

# Common Foundations for Recursive Shape Languages

23<sup>rd</sup> International Conference on Principles of Knowledge Representation and Reasoning

July 21<sup>st</sup> 2026 | Lisbon, Portugal

---

|                        |                                           |                   |                                     |
|------------------------|-------------------------------------------|-------------------|-------------------------------------|
| Shqiponja Ahmetaj      | (TU Wien, Austria)                        | Iovka Boneva      | (Université de Lille, France)       |
| Jan Hidders            | (Birkbeck, University of London, UK)      | Maxime Jakubowski | (TU Wien, Austria)                  |
| Jose-Emilio Labra-Gayo | (University of Oviedo, Spain)             | Wim Martens       | (University of Bayreuth, Germany)   |
| Fabio Mogavero         | (Università di Napoli Federico II, Italy) | Filip Murlak      | (University of Warsaw, Poland)      |
| <u>Cem Okulmus</u>     | (Paderborn University, Germany)           | Ognjen Savković   | (Free University of Bolzano, Italy) |
| Mantas Šimkus          | (TU Wien, Austria)                        | Dominik Tomaszuk  | (University of Białystok, Poland)   |

There are **large knowledge graphs** out there:

- DBPedia, Wikidata, Yago, Google Knowledge Graph, Amazon product graph

There are **large knowledge graphs** out there:

- DBPedia, Wikidata, Yago, Google Knowledge Graph, Amazon product graph

More and more realisation that **knowledge graphs need schemas**:

- tell users and systems what to expect in the data, limit data modifications;
- ad-hoc solutions deployed, multiple standards emerging

There are **large knowledge graphs** out there:

- DBPedia, Wikidata, Yago, Google Knowledge Graph, Amazon product graph

More and more realisation that **knowledge graphs need schemas**:

- tell users and systems what to expect in the data, limit data modifications;
- ad-hoc solutions deployed, multiple standards emerging

For this talk, we focus solely **on schema languages over RDF** graphs.

## Shapes Constraint Language (**SHACL**)

## Shape Expressions (**ShEx**)

## Shapes Constraint Language (**SHACL**)

## Shape Expressions (**ShEx**)

**Paradigm**

## Shapes Constraint Language (**SHACL**)

## Shape Expressions (**ShEx**)

### Paradigm

**Description logic** for  
specifying constraints  
+ regular path expressions

## Shapes Constraint Language (**SHACL**)

## Shape Expressions (**ShEx**)

### Paradigm

**Description logic** for  
specifying constraints  
+ regular path expressions

**Regular expressions**  
for neighbourhoods,  
close to XML-Schema



## Shapes Constraint Language (**SHACL**)

## Shape Expressions (**ShEx**)

### Paradigm

**Description logic** for  
specifying constraints  
+ regular path expressions

**Regular expressions**  
for neighbourhoods,  
close to XML-Schema

### Recursion

**Left Undefined!**

Greatest Fixpoint Semantics

## Shapes Constraint Language (**SHACL**)

## Shape Expressions (**ShEx**)

### Paradigm

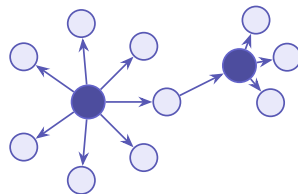
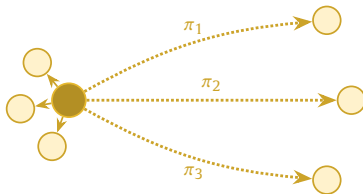
**Description logic** for  
specifying constraints  
+ regular path expressions

**Regular expressions**  
for neighbourhoods,  
close to XML-Schema

### Recursion

**Left Undefined!**

Greatest Fixpoint Semantics



## Three research questions:

1) What is the best semantics for recursion in SHACL?

## Three research questions:

1) What is the best semantics for recursion in SHACL?

2) How do the semantics of ShEx and *a recursive SHACL* compare ?

3) What semantics do real-world validators actually follow?

## Our Contributions:

## Three research questions:

- 1) What is the best semantics for recursion in SHACL?
- 2) How do the semantics of ShEx and *a recursive SHACL* compare ?
- 3) What semantics do real-world validators actually follow?

## Our Contributions:

- 1) Provide a **complete** overview of the data and combined complexity of ShEx & SHACL

## Three research questions:

- 1) What is the best semantics for recursion in SHACL?
- 2) How do the semantics of ShEx and *a recursive SHACL* compare ?
- 3) What semantics do real-world validators actually follow?

## Our Contributions:

- 1) Provide a **complete** overview of the data and combined complexity of ShEx & SHACL
- 2) Show that a (fragment) of ShEx with GFP and SHACL with LFP are equi-expressive.

## Three research questions:

- 1) What is the best semantics for recursion in SHACL?
- 2) How do the semantics of ShEx and *a recursive SHACL* compare ?
- 3) What semantics do real-world validators actually follow?

## Our Contributions:

- 1) Provide a **complete** overview of the data and combined complexity of ShEx & SHACL
- 2) Show that a (fragment) of ShEx with GFP and SHACL with LFP are equi-expressive.
- 3) Performed a small experimental study, aiming to understand what real-world validators do in **presence of recursion**.





s ... shape name

$s \dots$  shape name

$\varphi \dots$  shape expression

$s \dots$  shape name

$\varphi \dots$  shape expression

$s \leftarrow \varphi \dots$  shape assignment

$s \dots$  shape name

$\varphi \dots$  shape expression

$s \leftarrow \varphi \dots$  shape assignment

$\{s_1 \leftarrow \varphi_1, \dots, s_n \leftarrow \varphi_n\} \dots$  shape catalogue

$s \dots$  shape name

$\varphi \dots$  shape expression

$s \leftarrow \varphi \dots$  shape assignment

$\{s_1 \leftarrow \varphi_1, \dots, s_n \leftarrow \varphi_n\} \dots$  shape catalogue

Dependency graph:

$$\{s_1 \leftarrow \exists r. s_2 \wedge \exists t. (\neg s_3)\}$$

$s \dots$  shape name

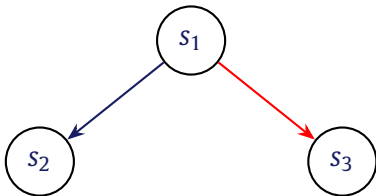
$\varphi \dots$  shape expression

$s \leftarrow \varphi \dots$  shape assignment

$\{s_1 \leftarrow \varphi_1, \dots, s_n \leftarrow \varphi_n\} \dots$  shape catalogue

Dependency graph:

$$\{s_1 \leftarrow \exists r.s_2 \wedge \exists t.(\neg s_3)\}$$



$s \dots$  shape name

$\varphi \dots$  shape expression

$s \leftarrow \varphi \dots$  shape assignment

$\{s_1 \leftarrow \varphi_1, \dots, s_n \leftarrow \varphi_n\} \dots$  shape catalogue

Dependency graph:

$$\{s_1 \leftarrow \exists r. s_2 \wedge \exists t. (\neg s_3), \quad s_2 \leftarrow \exists r. s_1\}$$

$s \dots$  shape name

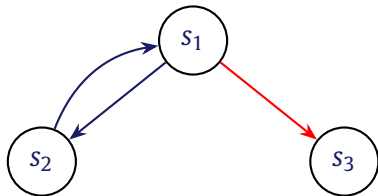
$\varphi \dots$  shape expression

$s \leftarrow \varphi \dots$  shape assignment

$\{s_1 \leftarrow \varphi_1, \dots, s_n \leftarrow \varphi_n\} \dots$  shape catalogue

Dependency graph:

$$\{s_1 \leftarrow \exists r.s_2 \wedge \exists t.(\neg s_3), \quad s_2 \leftarrow \exists r.s_1\}$$



A shape catalogue is **recursive** if its dependency graph has a cycle.



$s \dots$  shape name

$\varphi \dots$  shape expression

$s \leftarrow \varphi \dots$  shape assignment

$\{s_1 \leftarrow \varphi_1, \dots, s_n \leftarrow \varphi_n\} \dots$  shape catalogue

Dependency graph:

$$\{s_1 \leftarrow \exists r. s_2 \wedge \exists t. (\neg s_3), \quad s_2 \leftarrow \exists r. (\neg s_1)\}$$

$s \dots$  shape name

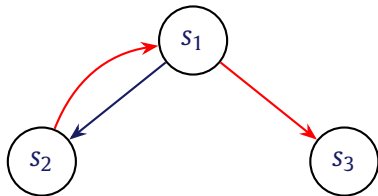
$\varphi \dots$  shape expression

$s \leftarrow \varphi \dots$  shape assignment

$\{s_1 \leftarrow \varphi_1, \dots, s_n \leftarrow \varphi_n\} \dots$  shape catalogue

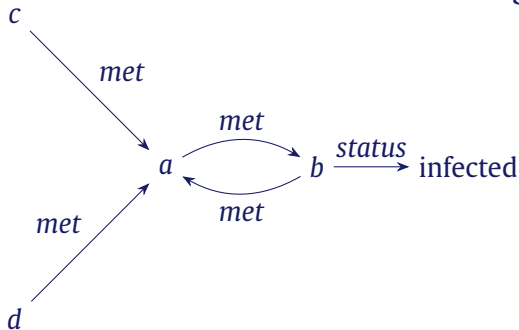
Dependency graph:

$$\{s_1 \leftarrow \exists r. s_2 \wedge \exists t. (\neg s_3), \quad s_2 \leftarrow \exists r. (\neg s_1)\}$$



A shape catalogue is **not stratified** if its dependency graph has a cycle involving a **negative** edge.

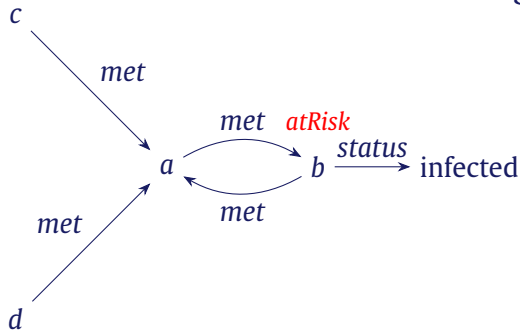
We are given the graph  $\mathcal{G}$  ...



... and a schema  $S$ .

$$S = \{atRisk: \exists status.infected \vee \exists met.atRisk\}$$

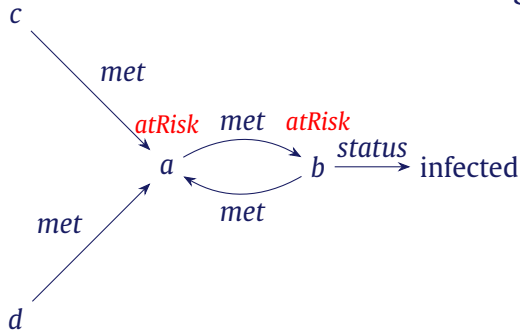
We are given the graph  $\mathcal{G}$  ...



... and a schema  $S$ .

$$S = \{atRisk: \exists status.infected \vee \exists met.atRisk\}$$

We are given the graph  $\mathcal{G}$  ...



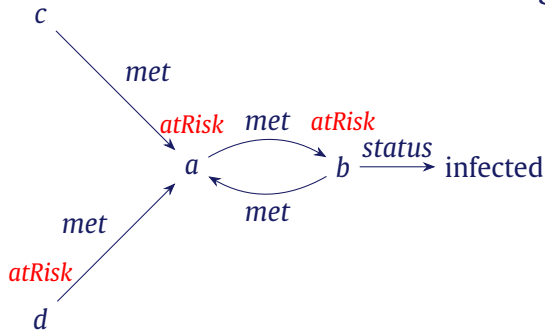
... and a schema  $S$ .

$$S = \{ \text{atRisk} : \exists \text{status}.\text{infected} \vee \exists \text{met}.\text{atRisk} \}$$

# Example - Recursive Schemas

We are given the graph  $\mathcal{G}$  ...

*atRisk*



... and a schema  $S$ .

$$S = \{ \textit{atRisk} : \exists \textit{status}.\textit{infected} \vee \exists \textit{met}.\textit{atRisk} \}$$

Define a **Simple Shape Language**.

Define a **Simple Shape Language**.

## Terminology

- Countable set of nodes  $N$



Define a **Simple Shape Language**.

## Terminology

- Countable set of nodes  $N$
- RDF graph  $\mathcal{G} \subseteq N \times N \times N$ , set of triples. Let  $\text{Nodes}(\mathcal{G})$  nodes occurring in  $\mathcal{G}$

Define a **Simple Shape Language**.

## Terminology

- Countable set of nodes  $N$
- RDF graph  $\mathcal{G} \subseteq N \times N \times N$ , set of triples. Let  $\text{Nodes}(\mathcal{G})$  nodes occurring in  $\mathcal{G}$
- Set of shape names  $S$

Define a **Simple Shape Language**.

## Terminology

- Countable set of nodes  $N$
- RDF graph  $\mathcal{G} \subseteq N \times N \times N$ , set of triples. Let  $\text{Nodes}(\mathcal{G})$  nodes occurring in  $\mathcal{G}$
- Set of shape names  $S$
- Shape assignment  $\alpha : N \rightarrow 2^S$

## Syntax & Semantics

## Syntax & Semantics

---

$$\varphi' \quad \llbracket \varphi' \rrbracket_{\mathcal{G}}^{\alpha} \subseteq N$$

## Syntax & Semantics

|            |                                                                     |
|------------|---------------------------------------------------------------------|
| $\varphi'$ | $\llbracket \varphi' \rrbracket_{\mathcal{G}}^{\alpha} \subseteq N$ |
| $\top$     | $\text{Nodes}(\mathcal{G})$                                         |

## Syntax & Semantics

|                  |                                                                     |
|------------------|---------------------------------------------------------------------|
| $\varphi'$       | $\llbracket \varphi' \rrbracket_{\mathcal{G}}^{\alpha} \subseteq N$ |
| $\top$           | $\text{Nodes}(\mathcal{G})$                                         |
| $\text{test}(u)$ | $\{u\}$                                                             |

## Syntax & Semantics

| $\varphi'$       | $\llbracket \varphi' \rrbracket_{\mathcal{G}}^{\alpha} \subseteq N$ |
|------------------|---------------------------------------------------------------------|
| $\top$           | $\text{Nodes}(\mathcal{G})$                                         |
| $\text{test}(u)$ | $\{u\}$                                                             |
| $s$              | $\{u \mid s \in \alpha(u)\}$                                        |



## Syntax & Semantics

| $\varphi'$       | $\llbracket \varphi' \rrbracket_{\mathcal{G}}^{\alpha} \subseteq N$                        |
|------------------|--------------------------------------------------------------------------------------------|
| $\top$           | $\text{Nodes}(\mathcal{G})$                                                                |
| $\text{test}(u)$ | $\{u\}$                                                                                    |
| $s$              | $\{u \mid s \in \alpha(u)\}$                                                               |
| $\neg\varphi$    | $\text{Nodes}(\mathcal{G}) \setminus \llbracket \varphi \rrbracket_{\mathcal{G}}^{\alpha}$ |

## Syntax & Semantics

| $\varphi'$                   | $\llbracket \varphi' \rrbracket_{\mathcal{G}}^{\alpha} \subseteq N$                                                  |
|------------------------------|----------------------------------------------------------------------------------------------------------------------|
| $\top$                       | $\text{Nodes}(\mathcal{G})$                                                                                          |
| $\text{test}(u)$             | $\{u\}$                                                                                                              |
| $s$                          | $\{u \mid s \in \alpha(u)\}$                                                                                         |
| $\neg\varphi$                | $\text{Nodes}(\mathcal{G}) \setminus \llbracket \varphi \rrbracket_{\mathcal{G}}^{\alpha}$                           |
| $\varphi_1 \wedge \varphi_2$ | $\llbracket \varphi_1 \rrbracket_{\mathcal{G}}^{\alpha} \cap \llbracket \varphi_2 \rrbracket_{\mathcal{G}}^{\alpha}$ |

## Syntax & Semantics

| $\varphi'$                   | $\llbracket \varphi' \rrbracket_{\mathcal{G}}^{\alpha} \subseteq N$                                                  |
|------------------------------|----------------------------------------------------------------------------------------------------------------------|
| $\top$                       | $\text{Nodes}(\mathcal{G})$                                                                                          |
| $\text{test}(u)$             | $\{u\}$                                                                                                              |
| $s$                          | $\{u \mid s \in \alpha(u)\}$                                                                                         |
| $\neg\varphi$                | $\text{Nodes}(\mathcal{G}) \setminus \llbracket \varphi \rrbracket_{\mathcal{G}}^{\alpha}$                           |
| $\varphi_1 \wedge \varphi_2$ | $\llbracket \varphi_1 \rrbracket_{\mathcal{G}}^{\alpha} \cap \llbracket \varphi_2 \rrbracket_{\mathcal{G}}^{\alpha}$ |
| $\exists p.\varphi$          | $\{u \mid \text{for some } (u, p, v) \in \mathcal{G}, v \in \llbracket \varphi \rrbracket_{\mathcal{G}}^{\alpha}\}$  |

## Syntax & Semantics

| $\varphi'$                   | $\llbracket \varphi' \rrbracket_{\mathcal{G}}^{\alpha} \subseteq N$                                                  |
|------------------------------|----------------------------------------------------------------------------------------------------------------------|
| $\top$                       | $\text{Nodes}(\mathcal{G})$                                                                                          |
| $\text{test}(u)$             | $\{u\}$                                                                                                              |
| $s$                          | $\{u \mid s \in \alpha(u)\}$                                                                                         |
| $\neg\varphi$                | $\text{Nodes}(\mathcal{G}) \setminus \llbracket \varphi \rrbracket_{\mathcal{G}}^{\alpha}$                           |
| $\varphi_1 \wedge \varphi_2$ | $\llbracket \varphi_1 \rrbracket_{\mathcal{G}}^{\alpha} \cap \llbracket \varphi_2 \rrbracket_{\mathcal{G}}^{\alpha}$ |
| $\exists p.\varphi$          | $\{u \mid \text{for some } (u, p, v) \in \mathcal{G}, v \in \llbracket \varphi \rrbracket_{\mathcal{G}}^{\alpha}\}$  |

$\alpha$  is a correct assignment for  $C$  if

$$\{u \mid s \in \alpha(u)\} = \llbracket \varphi \rrbracket_{\mathcal{G}}^{\alpha}$$

for every  $s \leftarrow \varphi$  in  $C$

$\alpha$  is a correct assignment for  $C$  if

$$\{u \mid s \in \alpha(u)\} = \llbracket \varphi \rrbracket_{\mathcal{G}}^{\alpha}$$

for every  $s \leftarrow \varphi$  in  $C$

For **general** catalogues

## Supported Model Semantics (SMS)

$\alpha$  conforms to  $C$  and  $\mathcal{G}$  under SMS iff  $\alpha$  is a correct assignment for  $C$ .

$\alpha$  is a correct assignment for  $C$  if

$$\{u \mid s \in \alpha(u)\} = \llbracket \varphi \rrbracket_{\mathcal{G}}^{\alpha}$$

for every  $s \leftarrow \varphi$  in  $C$

For **general** catalogues

## Supported Model Semantics (SMS)

$\alpha$  conforms to  $C$  and  $\mathcal{G}$  under SMS iff  $\alpha$  is a correct assignment for  $C$ .

For **stratified** catalogues

## Least Fixpoint Semantics (LFP)

$\alpha$  conforms to  $C$  and  $\mathcal{G}$  under LFP iff  $\alpha$  is the **smallest** correct assignment for  $C$ .

## Greatest Fixpoint Semantics (GFP)

$\alpha$  conforms to  $C$  and  $\mathcal{G}$  under GFP iff  $\alpha$  is the **greatest** correct assignment for  $C$ .

## Observation

As shown in WWW '25, both full SHACL and ShEx are mutually incomparable.



## Observation

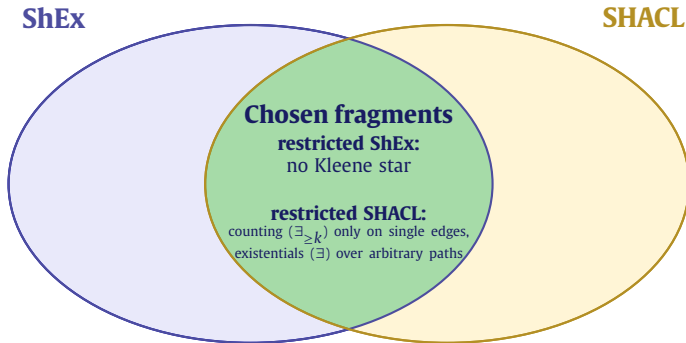
As shown in WWW '25, both full SHACL and ShEx are mutually incomparable.

⇒ find large fragment with same expressiveness

## Observation

As shown in WWW '25, both full SHACL and ShEx are mutually incomparable.

⇒ find large fragment with same expressiveness



GFP restricted ShEx  $\equiv$  LFP restricted SHACL

|       | Data complexity |       | Combined complexity |           |
|-------|-----------------|-------|---------------------|-----------|
|       | LFP/GFP         | SMS   | LFP/GFP             | SMS       |
| SHACL | $P^1$           | $P^1$ | $NP^2$              | $NP^2$    |
| ShEx  | $P$             | $NP$  | $P^{NP}$            | $NP^{NP}$ |

For SHACL, the complexity of validation was already known.

We completed the picture by showing that:

1. In the data complexity, ShEx and SHACL behave analogously and
2. In combined case for ShEx, there is a jump in complexity, needing an NP oracle.

<sup>1</sup>Andresel, Corman, Ortiz, Reutter, Savković, and Šimkus, “Stable model semantics for recursive SHACL”, WWW 2020

<sup>2</sup>Corman, Reutter, and Savković, “Semantics and validation of recursive SHACL”, ISWC 2018



Validation: Equipping shapes with targets

*A target assertion  $s(u)$*

Validation: Equipping shapes with targets

A *target assertion*  $s(u)$  is valid under  $C$  and  $\mathcal{G}$  for  $\alpha$  if  $u \in \llbracket \varphi \rrbracket_{\mathcal{G}}^{\alpha}$ , where  $s \leftarrow \varphi \in C$ .

Validation: Equipping shapes with targets

A *target assertion*  $s(u)$  is valid under  $C$  and  $\mathcal{G}$  for  $\alpha$  if  $u \in \llbracket \varphi \rrbracket_{\mathcal{G}}^{\alpha}$ , where  $s \leftarrow \varphi \in C$ .

Validation: Equipping shapes with targets

A *target assertion*  $s(u)$  is valid under  $C$  and  $\mathcal{G}$  for  $\alpha$  if  $u \in \llbracket \varphi \rrbracket_{\mathcal{G}}^{\alpha}$ , where  $s \leftarrow \varphi \in C$ .

**Problem:** how to determine the semantics from **just validation outcome alone**?



Validation: Equipping shapes with targets

A *target assertion*  $s(u)$  is valid under  $C$  and  $\mathcal{G}$  for  $\alpha$  if  $u \in \llbracket \varphi \rrbracket_{\mathcal{G}}^{\alpha}$ , where  $s \leftarrow \varphi \in C$ .

**Problem:** how to determine the semantics from **just validation outcome alone?**

**Solution:** **Separation Test** validating to true in LFP (resp. GFP) and false in the other.

Validation: Equipping shapes with targets

A *target assertion*  $s(u)$  is valid under  $C$  and  $\mathcal{G}$  for  $\alpha$  if  $u \in \llbracket \varphi \rrbracket_{\mathcal{G}}^{\alpha}$ , where  $s \leftarrow \varphi \in C$ .

**Problem:** how to determine the semantics from **just validation outcome alone?**

**Solution:** **Separation Test** validating to true in LFP (resp. GFP) and false in the other.

# LFP, GFP or SMS - What ShEx validators actually do (1/2)

| Test instance | rudof (ShEx) | Jena ShEx | ShEx-S | GFP | LFP | brave | SMS | cautious | SMS |
|---------------|--------------|-----------|--------|-----|-----|-------|-----|----------|-----|
| bsep1         |              |           |        | ●   | ■   | ●     |     |          | ■   |
| bsep2         |              |           |        | ●   | ■   | ●     |     |          | ■   |
| bsep3         |              |           |        | ●   | ■   | ●     |     |          | ■   |
| bsep4         |              |           |        | ●   | ■   | ●     |     |          | ■   |
| reach1        |              |           |        | ●   | ■   | ●     |     |          | ■   |
| reach2        |              |           |        | ■   | ●   | ●     |     |          | ■   |
| safe1         |              |           |        | ●   | ■   | ●     |     |          | ■   |
| safe2         |              |           |        | ●   | ■   | ●     |     |          | ■   |

●...pass

■...fail

✗ ...error

# LFP, GFP or SMS - What ShEx validators actually do (2/2)

| Test instance | rudof (ShEx) | Jena ShEx | ShEx-S | GFP | LFP | brave SMS | cautious SMS |
|---------------|--------------|-----------|--------|-----|-----|-----------|--------------|
| bsep1         | ●            | ●         | ●      | ●   | ■   | ●         | ■            |
| bsep2         | ●            | ●         | ●      | ●   | ■   | ●         | ■            |
| bsep3         | ●            | ●         | ●      | ●   | ■   | ●         | ■            |
| bsep4         | ●            | ●         | ●      | ●   | ■   | ●         | ■            |
| reach1        | ●            | ●         | ●      | ●   | ■   | ●         | ■            |
| reach2        | ■            | ■         | ■      | ■   | ●   | ●         | ■            |
| safe1         | ●            | ●         | ●      | ●   | ■   | ●         | ■            |
| safe2         | ●            | ●         | ●      | ●   | ■   | ●         | ■            |

●...pass      ■...fail      ✗ ...error

# LFP, GFP or SMS - What SHACL validators actually do

| Test instance | pySHACL | SHACL-S | Jena SHACL | Topbraid | rudof (SHACL) | GFP | LFP | bSMS | cSMS |
|---------------|---------|---------|------------|----------|---------------|-----|-----|------|------|
| bsep1         |         |         |            |          |               | ●   | ■   | ●    | ■    |
| bsep2         |         |         |            |          |               | ●   | ■   | ●    | ■    |
| bsep3         |         |         |            |          |               | ●   | ■   | ●    | ■    |
| bsep4         |         |         |            |          |               | ●   | ■   | ●    | ■    |
| reach1        |         |         |            |          |               | ●   | ■   | ●    | ■    |
| reach2        |         |         |            |          |               | ■   | ●   | ●    | ■    |
| safe1         |         |         |            |          |               | ●   | ■   | ●    | ■    |
| safe2         |         |         |            |          |               | ●   | ■   | ●    | ■    |

●...pass
■...fail
✗ ...error

# LFP, GFP or SMS - What SHACL validators actually do

| Test instance | pySHACL | SHACL-S | Jena SHACL | Topbraid | rudof (SHACL) | GFP | LFP | bSMS | cSMS |
|---------------|---------|---------|------------|----------|---------------|-----|-----|------|------|
| bsep1         | ●       | ●       | ●          | ●        | ●             | ●   | ■   | ●    | ■    |
| bsep2         | ●       | ●       | ●          | ●        | ●             | ●   | ■   | ●    | ■    |
| bsep3         | ✗       | ✗       | ●          | ■        | ✗             | ●   | ■   | ●    | ■    |
| bsep4         | ●       | ●       | ●          | ●        | ●             | ●   | ■   | ●    | ■    |
| reach1        | ●       | ●       | ●          | ■        | ●             | ●   | ■   | ●    | ■    |
| reach2        | ●       | ●       | ●          | ●        | ✗             | ■   | ●   | ●    | ■    |
| safe1         | ●       | ●       | ●          | ●        | ✗             | ●   | ■   | ●    | ■    |
| safe2         | ●       | ●       | ●          | ■        | ●             | ●   | ■   | ●    | ■    |

●...pass      ■...fail      ✗ ...error



Tool to recreate our experiments:  
[github.com/cogsl/sheval](https://github.com/cogsl/sheval)

Aiming to integrate more tests and validators in the future

- Explored what real-world validators for SHACL and ShEx actually do.



- Explored what real-world validators for SHACL and ShEx actually do.  
⇒ Standardisation leads to uniform behaviour (as expected)!

- Explored what real-world validators for SHACL and ShEx actually do.  
⇒ Standardisation leads to uniform behaviour (as expected)!
- Expressive power of SHACL with LFP and ShEx with GFP is equivalent.

- Explored what real-world validators for SHACL and ShEx actually do.  
⇒ Standardisation leads to uniform behaviour (as expected)!
- Expressive power of SHACL with LFP and ShEx with GFP is equivalent.
- Completed the complexity landscape: LFP and GFP permit tractable validation.

- Explored what real-world validators for SHACL and ShEx actually do.  
⇒ Standardisation leads to uniform behaviour (as expected)!
- Expressive power of SHACL with LFP and ShEx with GFP is equivalent.
- Completed the complexity landscape: LFP and GFP permit tractable validation.

- Explored what real-world validators for SHACL and ShEx actually do.  
⇒ Standardisation leads to uniform behaviour (as expected)!
- Expressive power of SHACL with LFP and ShEx with GFP is equivalent.
- Completed the complexity landscape: LFP and GFP permit tractable validation.

## Future Work

- Extend this work to (upcoming) SHACL 1.2 and its **rule sets**, extending the graph
- Exploring extensions of SHACL and ShEx to RDF 1.2